

FPGA Experiment Report

Experiment 6 — VGA Protocol (Screen Output)

Task: FPGA 6 — VGA interface for screen output

Student 1: Mahmood Stitia

ID: 214032682

Student 2: Saleh Khalil

ID: 213310485

Part 1 — Status update

The table below lists the submitted files and their status. The status is one of *Done* (module and simulation written and passing), *Partially done* (started but not fully working — explained in the notes), or *Not started*.

File / Step	Status	Notes / comments
VGA_Interface.v	Done	Timing / sync generator, 800×600 @ 72 Hz.
VGA_Interface_tb.v	In progress	Waveforms + analysis to be added (Part 2).
Drawer.v	Done	Movable rectangle; colour from <code>sw</code> .
Debouncer.v	Done	One clean pulse per button press.
VGA_Top.v	Done	Connects interface and drawer.
task2.xdc	Done	Basys-3 pin + clock constraints.

Issues

Any errors present at the time of submission are described here (per module / simulation, with what the error means).

- VGA_Interface.v: None.
- VGA_Interface_tb.v: Simulation pending.
- Drawer.v: None.
- Debouncer.v: None.
- VGA_Top.v: None.
- task2.xdc: None.

Part 2 — Simulations and answers

Simulation — VGA_Interface

The design targets the **800×600 @ 72 Hz** VGA mode driven by a 50 MHz pixel clock, generated by dividing the board's 100 MHz system clock by two. Two counters track the beam position: `h_count` runs 0 to 1039 (1040 pixel-clocks per line) and `v_count` runs 0 to 665 (666 lines per frame). Only the 800×600 window is visible; outside it the outputs are blanked to black.

Simulated Scenario

1. **Reset and counter roll-over.**
2. **Sync-pulse timing.**
3. **Visible vs. blanking colour gating.**

Pixel-clock and Blanking Considerations

The 50 MHz pixel rate is produced by toggling a register (`pclk`) on every system-clock edge, i.e. dividing the 100 MHz clock by two. The sync and colour outputs are registered (flip-flops) so the VGA pins stay glitch-free, and outside the 800×600 visible window the colour outputs are forced to $\{R, G, B\} = 0$ (blanking).

[paste screenshot here]

Full VGA_Interface_tb simulation with the TCL Console message [test result here].

Signals (top to bottom in the waveform):

- clk - 100 MHz system clock (test-bench driven).
- rstn - active-low reset.
- pclk - internal 50 MHz pixel clock (clk divided by two).
- pixel_color[11:0] - input colour word {R,G,B} from the Drawer.
- XCoord[10:0] / YCoord[10:0] - current pixel position.
- Hsync / Vsync - sync outputs (positive polarity).
- vgaRed/Green/Blue[3:0] - colour outputs (forced to 0 in blanking).

[paste screenshot here]

Annotated close-up of the same simulation. the sync edges and the visible-to-blanking transition.

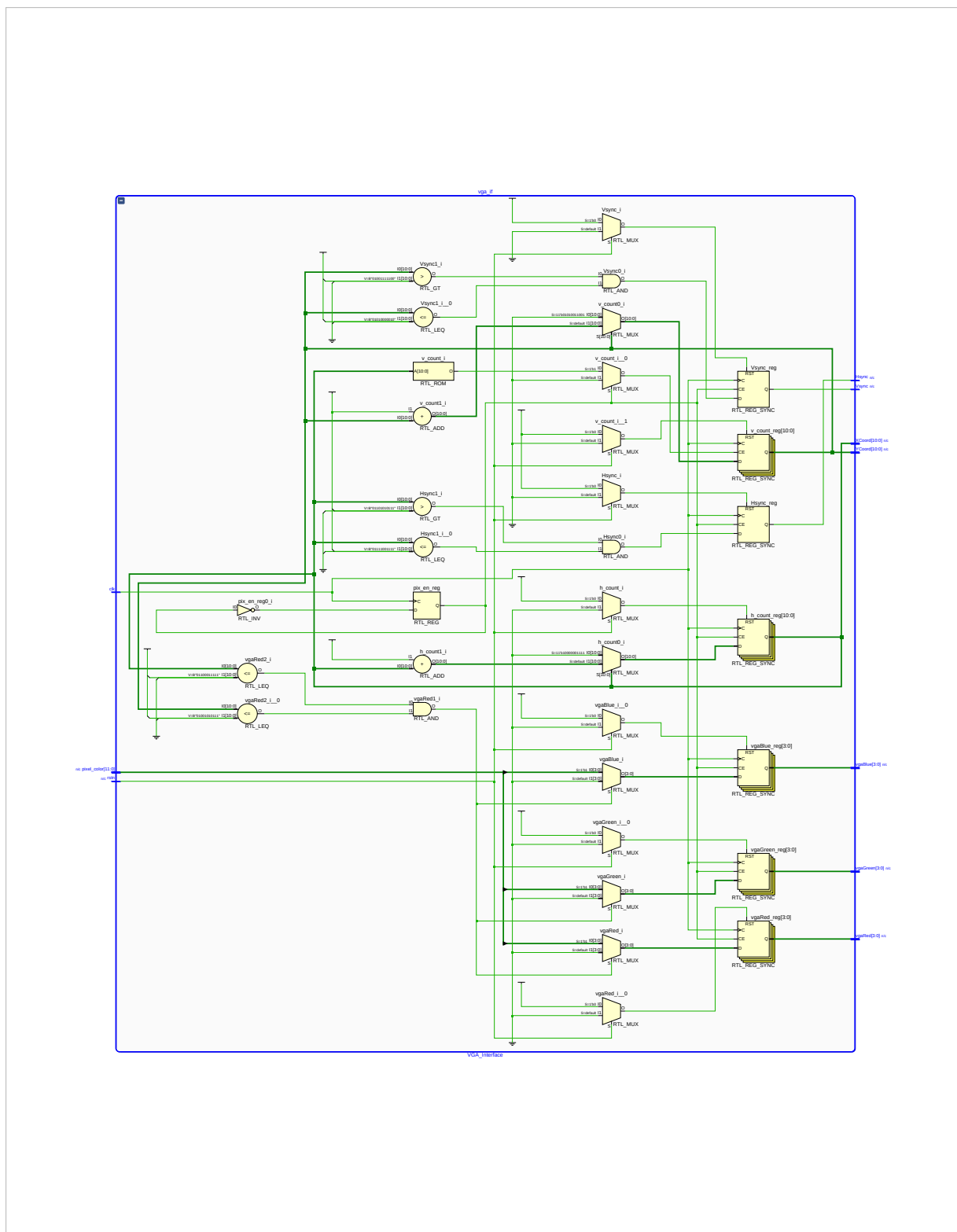
Elaborated Design

Top module

[paste screenshot here]

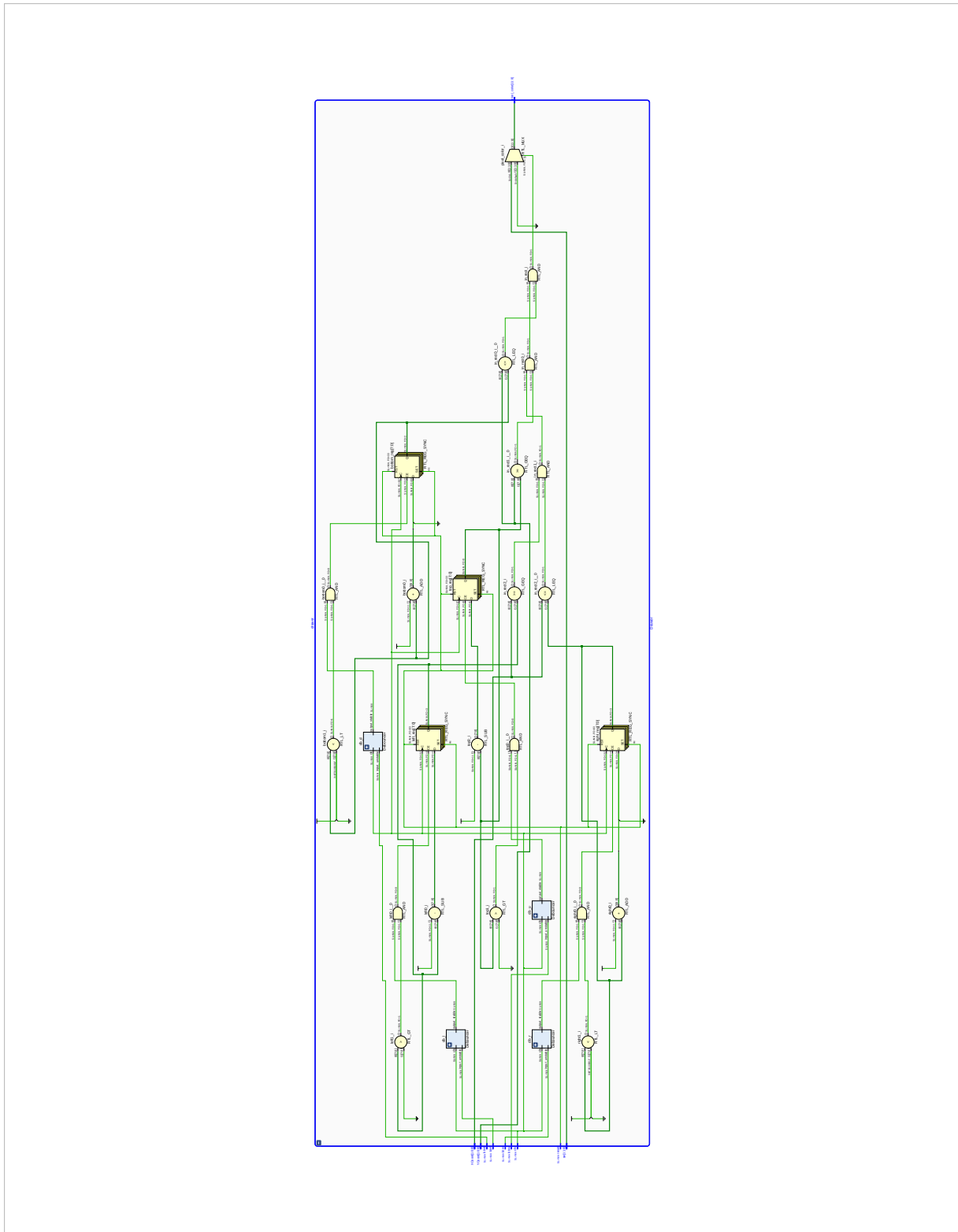
RTL-elaborated schematic of `VGA_Top`: `vga_if` (`VGA_Interface`) and `drawer` (`Drawer`), with the reset push-button inverted into `rstn`.

VGA_Interface



RTL-elaborated schematic of the VGA_Interface.

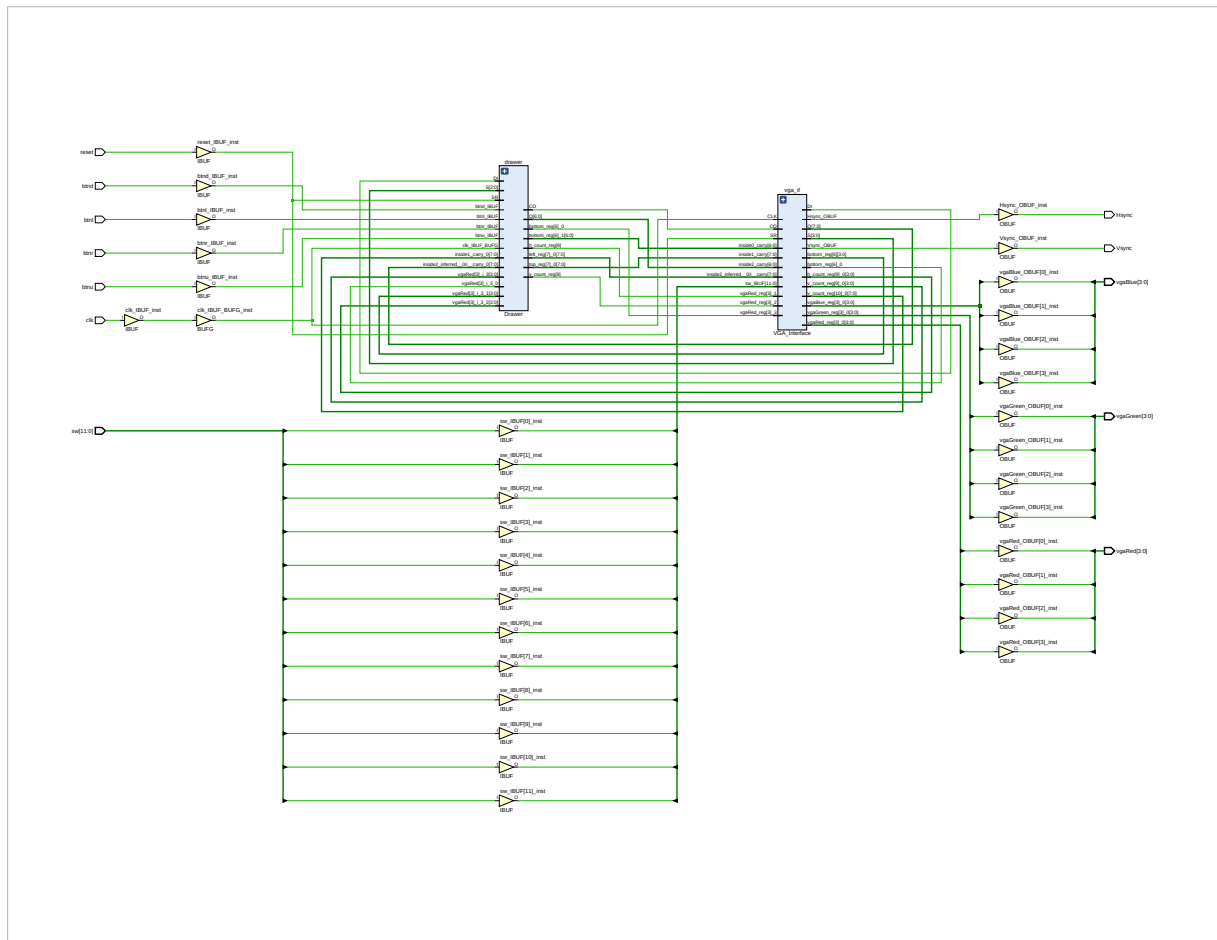
Drawer



RTL-elaborated schematic of the Drawer.

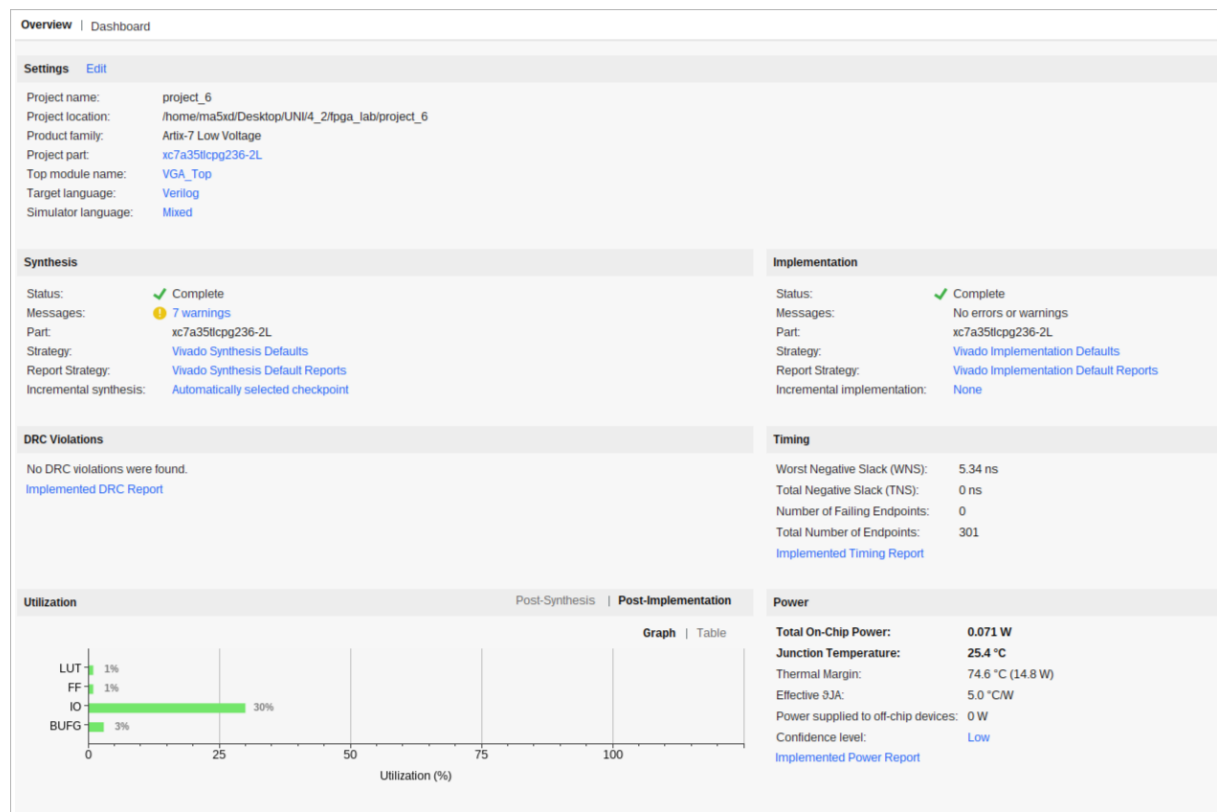
Implementation

Synthesis schematic



Post-synthesis schematic.

Project summary



Post-implementation project summary.

Timing summary

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5.340 ns		Worst Hold Slack (WHS): 0.236 ns		Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 301		Total Number of Endpoints: 301		Total Number of Endpoints: 156	
All user specified timing constraints are met.					

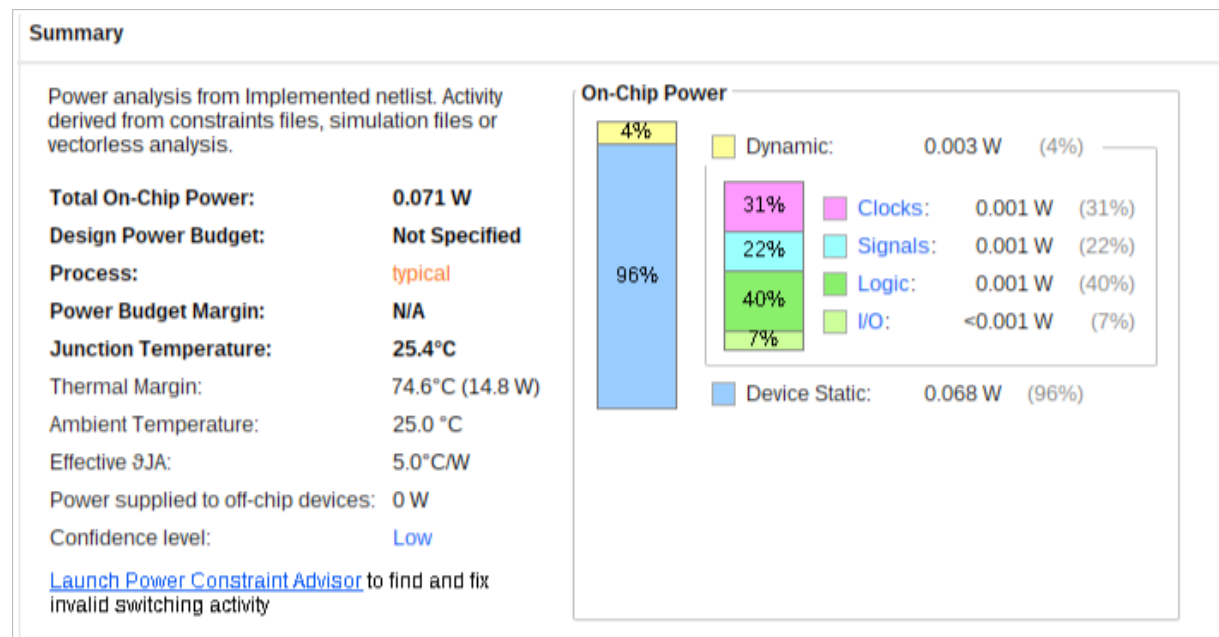
Design Timing Summary after implementation.

Critical Path

Name	Slack ^{^1}	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement
↳ Path 1	5.340	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[0]/D	4.613	1.753	2.860	10.0
↳ Path 2	5.354	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[1]/D	4.596	1.753	2.843	10.0
↳ Path 3	5.361	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[2]/D	4.592	1.753	2.839	10.0
↳ Path 4	5.368	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaRed_reg[3]/D	4.585	1.753	2.832	10.0
↳ Path 5	5.478	4	12	vga_if/h_count_reg[3]/C	vga_if/vgaBlue_reg[3]/D	4.473	1.753	2.720	10.0

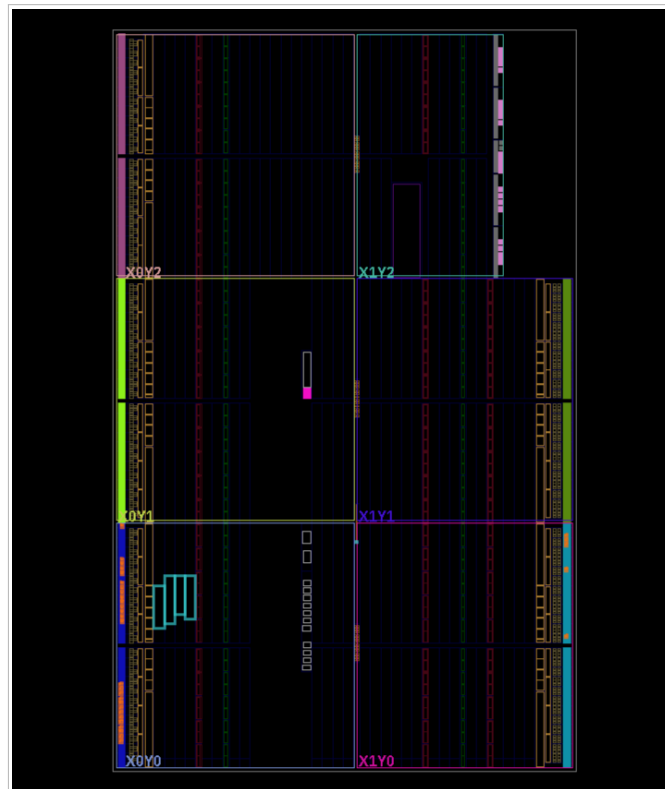
Intra-Clock Paths report (worst path, slack, logic levels and delay).

Power summary



Post-implementation power estimate.

Device



Device (floorplan) view of the placed-and-routed design on the Artix-7 (XC7A35T).

Problems we faced

Generating the 50 MHz pixel clock by dividing the system clock in fabric (the `pc1k` register) caused Vivado to flag every register it drives as a *non-clocked sequential cell* (TIMING-17), because a fabric-divided clock is not a recognised timing clock. Declaring it with `create_generated_clock` (alternatively, using a clock-enable instead of a second clock) makes those paths timed and clears the warnings.

[paste screenshot here]

The full system on hardware: the Basys-3 board driving the monitor over VGA, showing the movable rectangle (size/colour controlled by the switches and buttons).

Appendix — Verilog Source Code

The full source of the submitted files is listed below. Paste the code of each file inside its block.

VGA_Interface.v

```

1  `timescale 1ns/10ps
2  // Module Name: VGA_Interface - VGA timing/sync generator
3
4  `define INC(x, max) ((x == max-1)? 0 : (x + 1))
5
6  module VGA_Interface(
7      input wire      clk,
8      input wire      rstn,
9      input wire [11:0] pixel_color,
10     output reg [3:0] vgaRed,
11     output reg [3:0] vgaGreen,
12     output reg [3:0] vgaBlue,
13     output reg      Hsync,
14     output reg      Vsync,
15     output wire [10:0] XCoord,
16     output wire [10:0] YCoord
17 );
18
19     reg [10:0] h_count;
20     reg [10:0] v_count;
21
22     localparam H_VISIBLE = 799, H_FRONT_PORCH = 855, H_SYNC_END = 975,
23               H_TOTAL = 1039;
24     localparam V_VISIBLE = 599, V_FRONT_PORCH = 636, V_SYNC_END = 642,
25               V_TOTAL = 665;
26
27     // 50 MHz pixel-clock (divide 100 MHz by 2)
28     reg pclk = 0;
29     always @(posedge clk) pclk <= ~pclk;
30
31     // Display logic (registered outputs)
32     always @(posedge pclk) begin
33         if (!rstn) begin
34             Hsync <= 1'b0; Vsync <= 1'b0;
35             vgaRed <= 4'h0; vgaGreen <= 4'h0; vgaBlue <= 4'h0;
36         end else begin
37             Hsync <= (h_count > H_FRONT_PORCH) && (h_count <= H_SYNC_END);
38             Vsync <= (v_count > V_FRONT_PORCH) && (v_count <= V_SYNC_END);
39             if ((h_count <= H_VISIBLE) && (v_count <= V_VISIBLE)) begin
40                 vgaRed <= pixel_color[11:8];
41                 vgaGreen <= pixel_color[7:4];
42                 vgaBlue <= pixel_color[3:0];
43             end else begin
44                 vgaRed <= 4'h0; vgaGreen <= 4'h0; vgaBlue <= 4'h0;
45             end
46         end
47     end
48
49     // Coordinate counters
50     always @(posedge pclk) begin
51         if (!rstn) begin
52             h_count <= 0; v_count <= 0;
53         end else begin
54             h_count <= 'INC(h_count, H_TOTAL+1);
55             if (h_count == H_TOTAL)
56                 v_count <= 'INC(v_count, V_TOTAL+1);
57         end
58     end

```

```
56     end
57
58     assign XCoord = h_count;
59     assign YCoord = v_count;
60
61 endmodule
```

VGA_Interface_tb.v

```
1 // VGA_Interface_tb.v --- test bench to be added
```

Drawer.v

```

1  `timescale 1ns/10ps
2  // Module Name: Drawer - pixel color / movable rectangle logic
3
4  module Drawer(
5      input wire      clk,
6      input wire      reset,
7      input wire [11:0] sw,
8      input wire      btneu,
9      input wire      btnd,
10     input wire      btneu,
11     input wire      btneu,
12     input wire [10:0] XCoord,
13     input wire [10:0] YCoord,
14     output wire [11:0] pixel_color
15 );
16
17     // 800x600 visible region downsampled by 8
18     localparam integer GRID_W = 100;
19     localparam integer GRID_H = 75;
20     localparam integer CX      = GRID_W / 2; // 50
21     localparam integer CY      = GRID_H / 2; // 37
22
23     // 20-bit debouncer
24     wire btneu_pulse, btnd_pulse, btneu_pulse, btneu_pulse;
25     Debouncer #(.COUNTER_BITS(20)) db_u (.clk(clk), .input_unstable(btneu), .
        output_stable(btneu_pulse));
26     Debouncer #(.COUNTER_BITS(20)) db_d (.clk(clk), .input_unstable(btnd), .
        output_stable(btnd_pulse));
27     Debouncer #(.COUNTER_BITS(20)) db_l (.clk(clk), .input_unstable(btneu), .
        output_stable(btneu_pulse));
28     Debouncer #(.COUNTER_BITS(20)) db_r (.clk(clk), .input_unstable(btneu), .
        output_stable(btneu_pulse));
29
30     // Rectangle edges in coarse-grid units (inclusive)
31     reg [7:0] top, bottom, left, right;
32
33     always @(posedge clk) begin
34         if (reset) begin
35             top <= CY; bottom <= CY; left <= CX; right <= CX;
36         end else begin
37             if (btneu_pulse && top > 0) top <= top - 1'b1;
38             if (btnd_pulse && bottom < GRID_H - 1) bottom <= bottom + 1'b1;
39             if (btneu_pulse && left > 0) left <= left - 1'b1;
40             if (btneu_pulse && right < GRID_W - 1) right <= right + 1'b1;
41         end
42     end
43
44     // Coarse-grid coordinate (divide by 8)
45     wire [7:0] x_ds = XCoord[10:3];
46     wire [7:0] y_ds = YCoord[10:3];
47
48     wire inside = (x_ds >= left) && (x_ds <= right)
49                 && (y_ds >= top) && (y_ds <= bottom);
50
51     assign pixel_color = inside ? sw : 12'h000;
52
53 endmodule

```

Debouncer.v

```
1  `timescale 1ns/10ps
2  // Module Name: Debouncer - one clean 1-cycle pulse per stable button press
3
4  module Debouncer(clk, input_unstable, output_stable);
5
6      input clk, input_unstable;
7      output reg output_stable;
8
9      parameter COUNTER_BITS = 7;
10     reg prev_msb;
11     reg [COUNTER_BITS-1:0] counter; // Hysteresis counter
12
13     always @(posedge clk) begin
14         if (input_unstable == 1)
15             counter <= (counter < {COUNTER_BITS{1'b1}}) ? counter + 1 :
16                 counter;
17         else
18             counter <= (counter > {COUNTER_BITS{1'b0}}) ? counter - 1 :
19                 counter;
20
21         // 1-cycle pulse on the 0->1 transition of the counter MSB
22         prev_msb <= counter[COUNTER_BITS-1];
23         output_stable <= (~prev_msb) & counter[COUNTER_BITS-1];
24     end
25 endmodule
```

VGA_Top.v

```
1  `timescale 1ns/10ps
2  // Module Name: VGA_Top - connects VGA_Interface and Drawer
3
4  module VGA_Top(
5      input wire          clk,
6      input wire          reset,
7      input wire [11:0] sw,
8      input wire          btneu,
9      input wire          btnd,
10     input wire          btnl,
11     input wire          btnr,
12     output wire [3:0] vgaRed,
13     output wire [3:0] vgaGreen,
14     output wire [3:0] vgaBlue,
15     output wire          Hsync,
16     output wire          Vsync
17 );
18
19     wire [10:0] XCoord;
20     wire [10:0] YCoord;
21     wire [11:0] pixel_color;
22
23     VGA_Interface vga_if(
24         .clk(clk), .rstn(~reset), .pixel_color(pixel_color),
25         .vgaRed(vgaRed), .vgaGreen(vgaGreen), .vgaBlue(vgaBlue),
26         .Hsync(Hsync), .Vsync(Vsync),
27         .XCoord(XCoord), .YCoord(YCoord)
28     );
29
30     Drawer drawer(
31         .clk(clk), .reset(reset), .sw(sw),
32         .btneu(btneu), .btnd(btnd), .btnl(btnl), .btnr(btnr),
33         .XCoord(XCoord), .YCoord(YCoord),
34         .pixel_color(pixel_color)
35     );
36
37 endmodule
```


task2.xdc

```
1 # Clock (100 MHz, pin W5)
2 set_property PACKAGE_PIN W5 [get_ports clk]
3 set_property IOSTANDARD LVCMOS33 [get_ports clk]
4 create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [
  get_ports clk]
5
6 # Optional: declare the fabric-divided 50 MHz pixel clock so its FFs are
  timed
7 # create_generated_clock -name pclk \
8 #   -source [get_pins -hierarchical -filter {NAME =~ *pclk_reg/C}] \
9 #   -divide_by 2 [get_pins -hierarchical -filter {NAME =~ *pclk_reg/Q}]
10
11 # Pin assignments (all IOSTANDARD = LVCMOS33):
12 #   sw[11:0]      -> R3 T2 T3 V2 W13 W14 V15 W15 W17 W16 V16 V17
13 #   reset (btnC)  -> U18
14 #   btneu/btnd/btnl/btnr -> T18 / U17 / W19 / T17
15 #   vgaRed[3:0]   -> N19 J19 H19 G19
16 #   vgaGreen[3:0] -> D17 G17 H17 J17
17 #   vgaBlue[3:0]  -> J18 K18 L18 N18
18 #   Hsync -> P19 , Vsync -> R19
19 # (full Basys-3 master constraints in task2.xdc)
```